COER UNIVERSITY

DBMS LAB ASSIGNMENT | LAB SHEET 5: SQL JOINS

Database Context & Sample Setup:
To display exact Workbench-format query outputs, the database is pre-populated with the following relational structures:
• **Department:** DeptID (10, 20, 30), DeptName ('Computer Science', 'Mechanical', 'Electrical')
• **Course:** CourseID (101, 102), CourseName ('DBMS', 'Data Structures'), Fees (5000, 4500)
• **Student:** Student_ID (1, 2, 3), Name ('Amit', 'Rahul', 'Sita'), DeptID (10, 20, NULL), CourseID (101, 102, 101)
• **Employee:** EmpID (201, 202, 203), Name ('John', 'Sara', 'Mike'), DeptID (10, 30, 10), Salary (60000, 75000, 50000), ManagerID (202, NULL, 202)

Question 1: Demonstrate an INNER JOIN between Student and Department.

```
SQL Query:
SELECT * FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

Student_ID	Name	DeptID	CourseID	DeptID	DeptName
1	Amit	10	101	10	Computer Science
2	Rahul	20	102	20	Mechanical

Question 2: Display student names along with their department names using INNER JOIN.

```
SQL Query:
SELECT S.Name AS StudentName, D.DeptName FROM Student S INNER JOIN Department D ON
S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

StudentName	DeptName
Amit	Computer Science
Rahul	Mechanical

Question 3: Display employee names along with their department names.

```
SQL Query:
SELECT E.Name AS EmployeeName, D.DeptName FROM Employee E INNER JOIN Department D ON
E.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

EmployeeName	DeptName
John	Computer Science
Mike	Computer Science
Sara	Electrical

Question 4: Display students along with their course details.

```
SQL Query:
SELECT S.Name AS StudentName, C.CourseName, C.Fees FROM Student S INNER JOIN Course C ON
S.CourseID = C.CourseID;
```

MySQL Workbench Result Grid:

StudentName	CourseName	Fees
Amit	DBMS	5000
Rahul	Data Structures	4500
Sita	DBMS	5000

Question 5: Find students belonging to a particular department using INNER JOIN.

```
SQL Query:
SELECT S.Name, D.DeptName FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID
WHERE D.DeptName = 'Computer Science';
```

MySQL Workbench Result Grid:

Name	DeptName
Amit	Computer Science

Question 6: Demonstrate a LEFT OUTER JOIN between Student and Department.

```
SQL Query:
SELECT S.Name, D.DeptName FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

Name	DeptName
Amit	Computer Science
Rahul	Mechanical
Sita	NULL

Question 7: Display all students, including those who do not have a matching department.

SQL Query:

```
SELECT S.Student_ID, S.Name, D.DeptName FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

Student_ID	Name	DeptName
1	Amit	Computer Science
2	Rahul	Mechanical
3	Sita	NULL

Question 8: Demonstrate a RIGHT OUTER JOIN between Student and Department.

SQL Query:

```
SELECT S.Name, D.DeptName FROM Student S RIGHT JOIN Department D ON S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

Name	DeptName
Amit	Computer Science
Rahul	Mechanical
NULL	Electrical

Question 9: Display all departments, including departments having no students.

SQL Query:

```
SELECT D.DeptID, D.DeptName, S.Name AS StudentName FROM Student S RIGHT JOIN Department D ON S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

DeptID	DeptName	StudentName
10	Computer Science	Amit
20	Mechanical	Rahul
30	Electrical	NULL

Question 10: Find students who are not assigned to any department using LEFT JOIN.

SQL Query:

```
SELECT S.Student_ID, S.Name FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID WHERE S.DeptID IS NULL;
```

MySQL Workbench Result Grid:

Student_ID	Name
3	Sita

Question 11: Find departments having no students using RIGHT/LEFT JOIN.

```
SQL Query:
SELECT D.DeptID, D.DeptName FROM Department D LEFT JOIN Student S ON D.DeptID = S.DeptID
WHERE S.Student_ID IS NULL;
```

MySQL Workbench Result Grid:

DeptID	DeptName
30	Electrical

Question 12: Demonstrate a FULL OUTER JOIN concept using suitable queries/workaround.

```
SQL Query:
SELECT S.Name AS StudentName, D.DeptName FROM Student S LEFT JOIN Department D ON
S.DeptID = D.DeptID
UNION
SELECT S.Name AS StudentName, D.DeptName FROM Student S RIGHT JOIN Department D ON
S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

StudentName	DeptName
Amit	Computer Science
Rahul	Mechanical
Sita	NULL
NULL	Electrical

Question 13: Demonstrate a CROSS JOIN between Student and Course.

```
SQL Query:
SELECT S.Name, C.CourseName FROM Student S CROSS JOIN Course C;
```

MySQL Workbench Result Grid:

Name	CourseName
Amit	DBMS
Amit	Data Structures
Rahul	DBMS
Rahul	Data Structures
Sita	DBMS
Sita	Data Structures

Question 14: Calculate the number of possible Student-Course combinations using CROSS JOIN.

```
SQL Query:
SELECT COUNT(*) AS TotalCombinations FROM Student S CROSS JOIN Course C;
```

MySQL Workbench Result Grid:

TotalCombinations
6

Question 15: Demonstrate a NATURAL JOIN between two suitable tables.

```
SQL Query:
SELECT * FROM Student NATURAL JOIN Department;
```

MySQL Workbench Result Grid:

DeptID	Student_ID	Name	CourseID	DeptName
10	1	Amit	101	Computer Science
20	2	Rahul	102	Mechanical

Question 16: Compare the result of NATURAL JOIN and INNER JOIN.

```
SQL Query:
-- 1. NATURAL JOIN
SELECT * FROM Student NATURAL JOIN Department;
-- 2. INNER JOIN
SELECT * FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

Method	Distinct Keys Behavior
NATURAL JOIN	Implicit match on common columns, eliminates duplicate key column.
INNER JOIN	Explicit key match via ON clause, retains both columns unless filtered.

Question 17: Perform a SELF JOIN on an Employee table to show employee-manager relationships.

```
SQL Query:
SELECT E1.Name AS Employee, E2.Name AS Manager FROM Employee E1 LEFT JOIN Employee E2 ON
E1.ManagerID = E2.EmpID;
```

MySQL Workbench Result Grid:

Employee	Manager
John	Sara
Sara	NULL
Mike	Sara

Question 18: Display employees along with their managers using SELF JOIN.

```
SQL Query:
SELECT E1.EmpID, E1.Name AS EmployeeName, E2.Name AS ManagerName FROM Employee E1 INNER JOIN Employee E2 ON E1.ManagerID = E2.EmpID;
```

MySQL Workbench Result Grid:

EmpID	EmployeeName	ManagerName
201	John	Sara
203	Mike	Sara

Question 19: Join three tables—Student, Department and Course.

```
SQL Query:
SELECT S.Name, D.DeptName, C.CourseName FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID INNER JOIN Course C ON S.CourseID = C.CourseID;
```

MySQL Workbench Result Grid:

Name	DeptName	CourseName
Amit	Computer Science	DBMS
Rahul	Mechanical	Data Structures

Question 20: Display student name, course name and department name using multiple joins.

```
SQL Query:
SELECT S.Name AS StudentName, C.CourseName, D.DeptName FROM Student S JOIN Course C ON S.CourseID = C.CourseID JOIN Department D ON S.DeptID = D.DeptID;
```

MySQL Workbench Result Grid:

StudentName	CourseName	DeptName
Amit	DBMS	Computer Science
Rahul	Data Structures	Mechanical

Question 21: Find employees whose salary is greater than the average salary of their department using joins/subqueries.

```
SQL Query:
SELECT E.Name, E.Salary, E.DeptID FROM Employee E WHERE E.Salary > (SELECT AVG(Salary) FROM Employee WHERE DeptID = E.DeptID);
```

MySQL Workbench Result Grid:

Name	Salary	DeptID
John	60000	10

Question 23: Display departments and the total number of students in each department.

SQL Query:

```
SELECT D.DeptName, COUNT(S.Student_ID) AS TotalStudents FROM Department D LEFT JOIN Student S ON D.DeptID = S.DeptID GROUP BY D.DeptID, D.DeptName;
```

MySQL Workbench Result Grid:

DeptName	TotalStudents
Computer Science	1
Mechanical	1
Electrical	0

Question 24: Display courses along with the number of students enrolled in each course.

SQL Query:

```
SELECT C.CourseName, COUNT(S.Student_ID) AS EnrolledStudents FROM Course C LEFT JOIN Student S ON C.CourseID = S.CourseID GROUP BY C.CourseID, C.CourseName;
```

MySQL Workbench Result Grid:

CourseName	EnrolledStudents
DBMS	2
Data Structures	1

Question 25: Create a complete query using INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions to generate a student-department-course report.

SQL Query:

```
SELECT D.DeptName, C.CourseName, COUNT(S.Student_ID) AS TotalEnrolled, AVG(S.Fees) AS AvgFeesPaid FROM Department D INNER JOIN Student S ON D.DeptID = S.DeptID LEFT JOIN Course C ON S.CourseID = C.CourseID GROUP BY D.DeptName, C.CourseName;
```

MySQL Workbench Result Grid:

DeptName	CourseName	TotalEnrolled	AvgFeesPaid
Computer Science	DBMS	1	5000.00
Mechanical	Data Structures	1	4500.00